Docker实战应用（一）

什么是Docker

Docker 最初是 dotCloud 公司创始人 Solomon Hykes 在法国期间发起的一个公司内部项目，它是基于 dotCloud 公司多年云服务技术的一次革新，并于 2013 年 3 月以 Apache 2.0 授权协议开源，主要项目代码在 GitHub 上进行维护。

Docker 自开源后受到广泛的关注和讨论，至今其 GitHub 项目已经超过 4 万 6 千个星标和一万多个 fork。甚至由于 Docker 项目的火爆，在 2013 年底，dotCloud 公司决定改名为Docker。

Docker 使用 Google 公司推出的 Go 语言 进行开发实现，基于 Linux 内核的cgroup，namespace，以及Union FS 等技术，对进程进行封装隔离，属于操作系统层面的***虚拟化技术***。由于隔离的进程独立于宿主和其它的隔离的进程，因此也称其为***容器***。

官方的说法：

> Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code quickly, you can significantly reduce the delay between writing code and running it in production.
>
> Docker是一个用于开发，交付和运行应用程序的开放平台。 Docker使您能够将应用程序与基础架构分开，从而可以快速交付软件。借助Docker，您可以以与管理应用程序相同的方式来管理基础架构。通过利用Docker快速交付，测试和部署代码的方法，您可以大大减少编写代码和在生产环境中运行代码之间的延迟。

总之：Docker是一种容器技术，它能够解决软件安装和迁移过程中环境问题。

为什么要使用Docker

作为一种新兴的虚拟化方式，`Docker`跟传统的虚拟化方式相比具有众多的优势。



传统虚拟机技术是虚拟出一套硬件后，在其上运行一个完整操作系统，在该系统上再运行所需应用进程



容器内的应用进程直接运行于宿主的内核，容器内没有自己的内核，而且也没有进行硬件虚拟。因此容器要比传统虚拟机更为轻便。

更高效的利用系统资源

由于容器不需要进行硬件虚拟以及运行完整操作系统等额外开销，`Docker` 对系统资源的利用率更高。无论是应用执行速度、内存损耗或者文件存储速度，都要比传统虚拟机技术更高效。因此，相比虚拟机技术，一个相同配置的主机，往往可以运行更多数量的应用。

更快速的启动时间

传统的虚拟机技术启动应用服务往往需要数分钟，而 `Docker` 容器应用，由于直接运行于宿主内核，无需启动完整的操作系统，因此可以做到秒级、甚至毫秒级的启动时间。大大的节约了开发、测试、部署的时间。

一致的运行环境

开发过程中一个常见的问题是环境一致性问题。由于开发环境、测试环境、生产环境不一致，导致有些 bug 并未在开发过程中被发现。而 `Docker` 的镜像提供了除内核外完整的运行时环境，确保了应用运行环境一致性，从而不会再出现 ***「这段代码在我机器上没问题啊」*** 这类问题。

持续交付和部署

对开发和运维人员来说，最希望的就是一次创建或配置，可以在任意地方正常运行。

使用 `Docker` 可以通过定制应用镜像来实现持续集成、持续交付、部署。开发人员可以通过 `Dockerfile` 来进行镜像构建，并结合持续集成(Continuous Integration)系统进行集成测试，而运维人员则可以直接在生产环境中快速部署该镜像，甚至结合持续交付(Continuous Delivery) 系统进行自动部署。

而且使用 `Dockerfile` 使镜像构建透明化，不仅仅开发团队可以理解应用运行环境，也方便运维团队理解应用运行所需条件，帮助更好的生产环境中部署该镜像。

更轻松的迁移

由于 `Docker` 确保了执行环境的一致性，使得应用的迁移更加容易。`Docker` 可以在很多平台上运行，无论是物理机、虚拟机、公有云、私有云，甚至是笔记本，其运行结果是一致的。因此用户可以很轻易的将在一个平台上运行的应用，迁移到另一个平台上，而不用担心运行环境的变化导致应用无法正常运行的情况。

更轻松的维护和扩展

`Docker` 使用的分层存储以及镜像的技术，使得应用重复部分的复用更为容易，也使得应用的维护更新更加简单，基于基础镜像进一步扩展镜像也变得非常简单。此外，`Docker` 团队同各个开源项目团队一起维护了一大批高质量的 官方镜像，既可以直接在生产环境使用，又可以作为基础进一步定制，大大的降低了应用服务的镜像制作成本。

对比传统虚拟机总结

特性	容器	虚拟机
启动	秒级	分钟级
硬盘使用	一般为 `MB`	一般为 `GB`
性能	接近原生	弱于
系统支持量	单机支持上千个容器	一般几十个

安全性

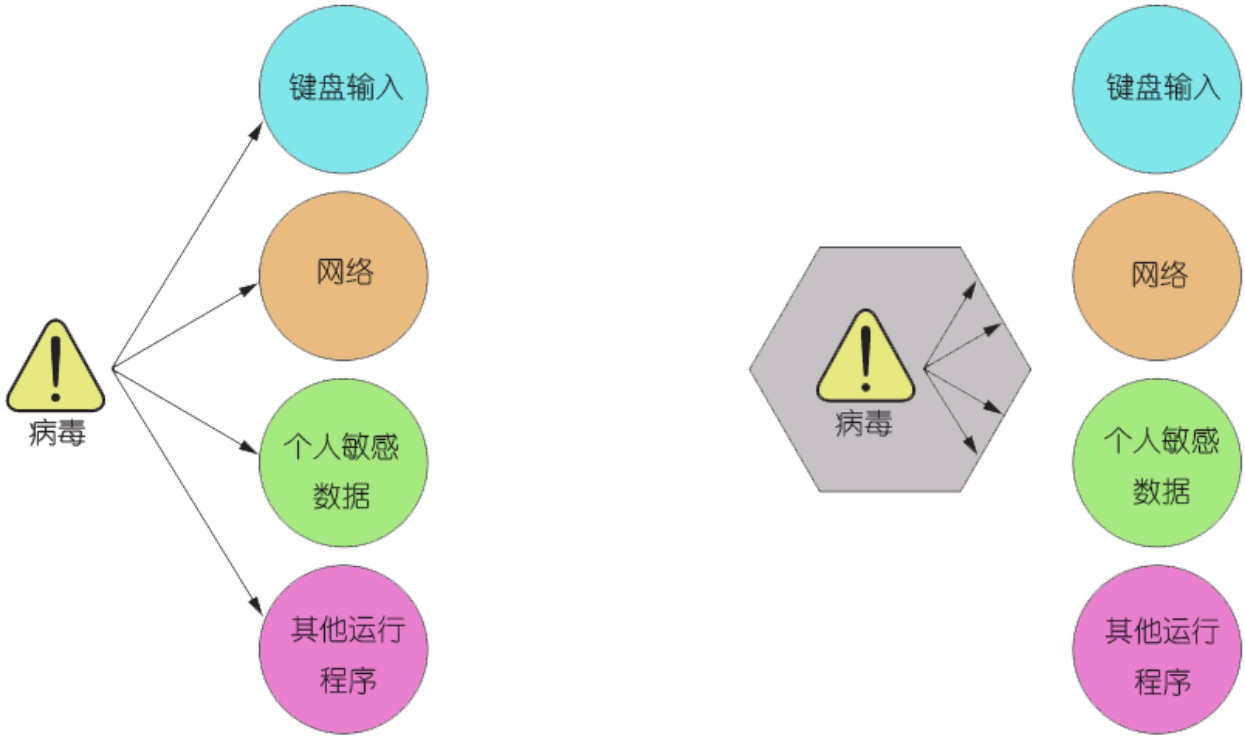
Docker的使用还有一个重要原因，那就是安全性问题。原因有三：

1）一个程序可能遇到特定的攻击。

2）牛逼而资深的程序员写出来的程序它副作用也很牛逼。

3）程序因为意外，也可以变成具有攻击的bug程序。

通过下图，我们理解一下有容器和没有容器的运行环境的安全性：

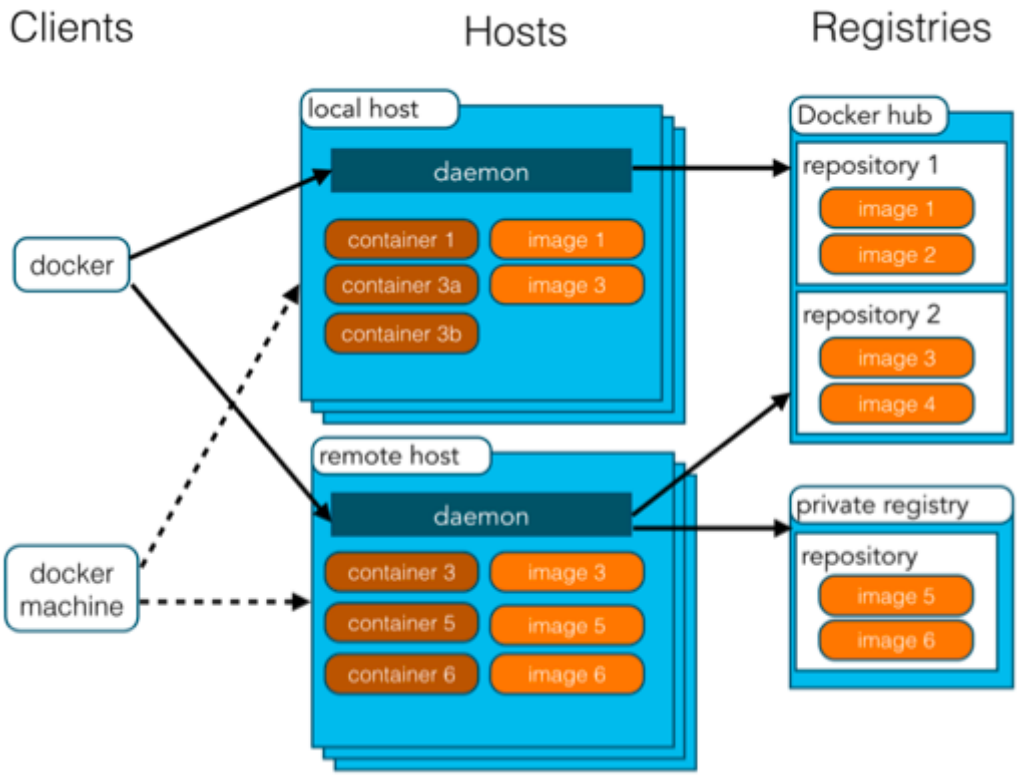


Docker架构

Docker 包括三个基本概念：

- 镜像（Image）
- 容器（Container）
- 仓库（Repository）

Docker 使用客户端-服务器 (C/S) 架构模式，使用远程API来管理和创建Docker容器。



Docker 镜像

我们都知道，操作系统分为内核和用户空间。对于 Linux 而言，内核启动后，会挂载 root文件系统为其提供用户空间支持。

而 Docker 镜像（Image），就相当于是一个 root 文件系统。比如官方镜像 ubuntu:16.04 就包含了完整的一套Ubuntu16.04 最小系统的 root 文件系统。

Docker 镜像是一个特殊的文件系统，除了提供容器运行时所需的程序、库、资源、配置等文件外，还包含了一些为运行时准备的一些配置参数（如匿名卷、环境变量、用户等）。镜像不包含任何动态数据，其内容在构建之后也不会被改变。

分层存储

因为镜像包含操作系统完整的 root 文件系统，其体积往往是庞大的，因此在 Docker 设计时，就充分利用 Union FS 的技术，将其设计为分层存储的架构。

所以严格来说，镜像并非是像一个 ISO 那样的打包文件，镜像只是一个虚拟的概念，其实际体现并非由一个文件组成，而是由一组文件系统组成，或者说，由多层文件系统联合组成。
镜像构建时，会一层层构建，前一层是后一层的基础。每一层构建完就不会再发生改变，后一层上的任何改变只发生在自己这一层。比如，删除前一层文件的操作，实际不是真的删除前一层的文件，而是仅在当前层标记为该文件已删除。在最终容器运行的时候，虽然不会看到这个文件，但是实际上该文件会一直跟随镜像。因此，在构建镜像的时候，需要额外小心，每一层尽量只包含该层需要添加的东西，任何额外的东西应该在该层构建结束前清理掉。

分层存储的特征还使得镜像的复用、定制变的更为容易。甚至可以用之前构建好的镜像作为基础层，然后进一步添加新的层，以定制自己所需的内容，构建新的镜像。
关于镜像构建，将会在后续相关章节中做进一步的讲解。

Docker 容器

镜像（`Image`）和容器（`Container`）的关系，就像是面向对象程序设计中的 `类` 和 `实例` 一样，镜像是静态的定义，容器是镜像运行时的实体。容器可以被创建、启动、停止、删除、暂停等。

前面讲过镜像使用的是分层存储，容器也是如此。每一个容器运行时，是以镜像为基础层，在其上创建一个当前容器的存储层，我们可以称这个为容器运行时读写而准备的存储层为 **容器存储层**。

容器存储层的生存周期和容器一样，容器消亡时，容器存储层也随之消亡。因此，任何保存于容器存储层的信息都会随容器删除而丢失。

按照 Docker 最佳实践的要求，容器不应该向其存储层内写入任何数据，容器存储层要保持无状态化。所有的文件写入操作，都应该使用 数据卷（Volume）、或者绑定宿主目录，在这些位置的读写会跳过容器存储层，直接对宿主（或网络存储）发生读写，其性能和稳定性更高。

数据卷的生存周期独立于容器，容器消亡，数据卷不会消亡。因此，使用数据卷后，容器删除或者重新运行之后，数据却不会丢失。

Docker 仓库

仓库可看成一个代码控制中心，用来保存镜像。

镜像构建完成后，可以很容易的在当前宿主机上运行，但是，如果需要在其它服务器上使用这个镜像，我们就需要一个集中的存储、分发镜像的服务，**Docker 仓库**就是这样的服务。

一个 **Docker 仓库** 中可以包含多个 **仓库**（`Repository`）；每个仓库可以包含多个 **标签**（`Tag`）；每个标签对应一个镜像。

通常，一个仓库会包含同一个软件不同版本的镜像，而标签就常用于对应该软件的各个版本。我们可以通过 `<仓库名>:<标签>` 的格式来指定具体是这个软件哪个版本的镜像。如果不给出标签，将以 `latest` 作为默认标签。

以 Ubuntu 镜像 为例，`ubuntu` 是仓库的名字，其内包含有不同的版本标签，如，`16.04`, `18.04`。我们可以通过 `ubuntu:16.04`，或者 `ubuntu:18.04` 来具体指定所需哪个版本的镜像。如果忽略了标签，比如 `ubuntu`，那将视为 `ubuntu:latest`。

公有 Docker 仓库服务

Docker 仓库公开服务是开放给用户使用、允许用户管理镜像的仓库服务。一般这类公开服务允许用户免费上传、下载公开的镜像，并可能提供收费服务供用户管理私有镜像。

最常使用的 Registry 公开服务是官方的 Docker Hub，这也是默认的 Registry，并拥有大量的高质量的官方镜像。除此以外，还有 Red Hat 的 Quay.io；Google 的 Google Container Registry，Kubernetes 的镜像使用的就是这个服务。

由于某些原因，在国内访问这些服务可能会比较慢。国内的一些云服务商提供了针对 Docker Hub 的镜像服务（`Registry Mirror`），这些镜像服务被称为 **加速器**。常见的有 阿里云加速器、DaoCloud 加速器 等。使用加速器会直接从国内的地址下载 Docker Hub 的镜像，比直接从 Docker Hub 下载速度会提高很多。

国内也有一些云服务商提供类似于 Docker Hub 的公开服务。比如 网易云镜像服务、DaoCloud 镜像市场、阿里云镜像库 等。

私有 Docker 仓库服务

除了使用公开服务外，用户还可以在本地搭建私有 Docker 仓库。Docker 官方提供了 Docker Registry 镜像，可以直接使用做为私有 Registry 服务。

开源的 Docker 仓库镜像只提供了 Docker Registry API 的服务端实现，足以支持 `docker` 命令，不影响使用。但不包含图形界面，以及镜像维护、用户管理、访问控制等高级功能。在官方的商业化版本 Docker Trusted Registry 中，提供了这些高级功能。

除了官方的 Docker Registry 外，还有第三方软件实现了 Docker Registry API，甚至提供了用户界面以及一些高级功能。比如，Harbor 和 Sonatype Nexus。

Docker 客户端(Client)

Docker 客户端通过命令行或者其他工具使用 Docker SDK (https://docs.docker.com/develop/sdk/) 与 Docker 的守护进程通信。

Docker 主机(Host)

一个物理或者虚拟的机器用于执行 Docker 守护进程和容器。

Docker Machine

Docker Machine是一个简化Docker安装的命令行工具，通过一个简单的命令行即可在相应的平台上安装Docker，比如VirtualBox、 Digital Ocean、Microsoft Azure。

Docker安装

安装参考：https://docs.docker.com/engine/install/centos/

安装最新版

```
# 1、卸载旧版本的Docker
dnf remove docker \
docker-client \
docker-client-latest \
docker-common \
docker-latest \
docker-latest-logrotate \
docker-logrotate \
docker-engine
# 2、yum源更新到最新
dnf update
# 3、安装需要的软件包
dnf -y install dnf-plugins-core
# 4、设置yum国内安装镜像源
dnf config-manager --add-repo \
https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
# 官方镜像：https://download.docker.com/linux/centos/docker-ce.repo
# 阿里镜像：https://mirrors.aliyun.com/docker-ce/linux/centos/docker-ce.repo
# 如果龙蜥系统高于8(如23.x)需要执行下面的命令，不然会找不到源
#sed -i 's/\$releasever/8/g' /etc/yum.repos.d/docker-ce.repo
# 5、更新软件源缓存
dnf makecache
# 6、安装最新版docker，如果想要安装指定版本，参考下面的指定版本安装步骤执行后续安装
dnf install -y \
docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
# 7、查看docker版本，验证是否验证成功
docker -v
```

安装成功截图，下面是查看版本号。

```
已安装:
  container-selinux-2:2.210.0-1.an23.noarch    containerd.io-1.6.32-3.1.el8.x86_6
  docker-compose-plugin-2.27.0-1.el8.x86_64    fuse-common-3.16.2-2.an23.x86_64
  slirp4netns-1.3.1-1.an23.x86_64

完毕!
[root@7c10c9d2eb8d home]# docker -v
Docker version 26.1.3, build b72abbb
[root@7c10c9d2eb8d home]#
```

启动Docker

```
# 启动Docker命令
systemctl start docker
# 查看启动结果
systemctl status docker
```

```
[root@localhost ~]# systemctl start docker
[root@localhost ~]# systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; vendor pres
et: disabled)
   Active: active (running) since
     Docs: https://docs.docker.com
 Main PID: 20513 (dockerd)
    Tasks: 11
   Memory: 34.0M
   CGroup: /system.slice/docker.service
           └─20513 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/con...
```

卸载Docker

如果你想要卸载Docker引擎，可以执行下面的命令

```
dnf remove docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin
```

主机上的镜像、容器、卷或自定义配置文件不会自动删除。要删除所有镜像、容器和卷，执行下面的命令：

```
rm -rf /var/lib/docker # 注意如果修改了docker-root的话，这里要调整哦
rm -rf /var/lib/containerd
```

您必须手动删除任何已编辑的配置文件。

Docker镜像加速器配置

国内从 Docker Hub 拉取镜像有时会遇到困难，此时可以配置镜像加速器。国内很多云服务商都提供了国内加速器服务，例如：

- 网易云加速器 `https://hub-mirror.c.163.com`
- 百度云加速器 `https://mirror.baidubce.com`
- DAOCloud `https://github.com/DaoCloud/public-image-mirror`
- 阿里云加速器(需登录账号获取)
- 第三方镜像服务参考

配置镜像

在 `/etc/docker/daemon.json` 中写入如下内容（如果文件不存在请新建该文件）

```
#如果文件不存在 新建文件
touch daemon.json
# 编辑文件 vi deamon.json  添加{}下面的内容
{
  "registry-mirrors": [
    "https://hub-mirror.c.163.com",
    "https://docker.m.daocloud.io",
    "https://noohub.ru",
    "https://huecker.io",
    "https://dockerhub.timeweb.cloud"
  ]
}
# 如果你还需要调整docker镜像存储位置，可以补充这个配置
"data-root": "/home/docker-root"
```

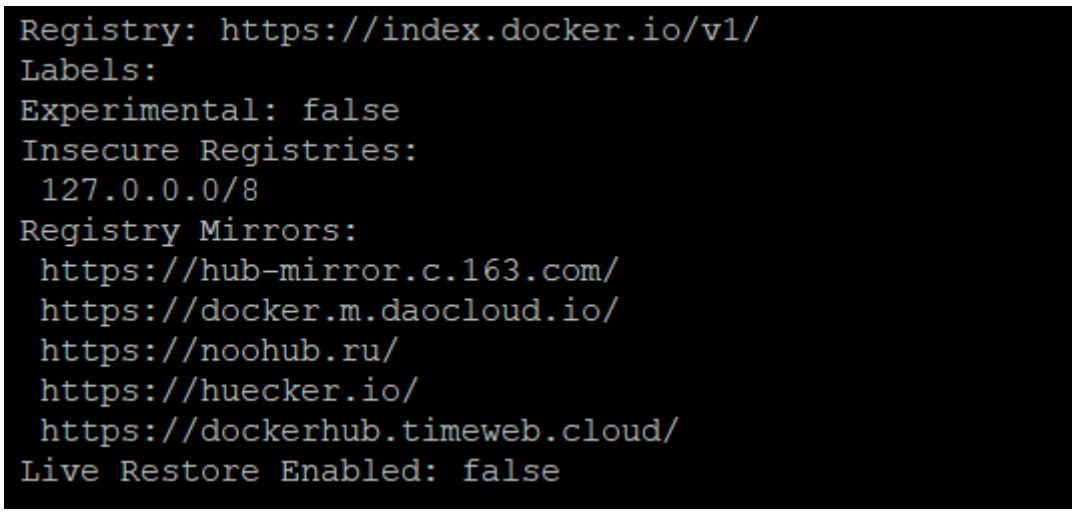
注意，一定要保证该文件符合 json 规范，否则 Docker 将不能启动。

重启服务

```
# 重新加载配置
systemctl daemon-reload
# 重启docker服务
systemctl restart docker
# 查看docker状态
systemctl status docker
```

检查加速器是否生效

执行 `docker info`，如果从结果中看到了你配置的镜像地址，说明配置成功。



阿里云镜像获取方式



云服务商

某些云服务商提供了仅供内部访问的镜像服务，当您的 Docker 运行在云平台时可以选择它们。

- Azure 中国镜像 `https://dockerhub.azk8s.cn`
- 腾讯云 `https://mirror.ccs.tencentyun.com`

Docker远程访问

开启访问

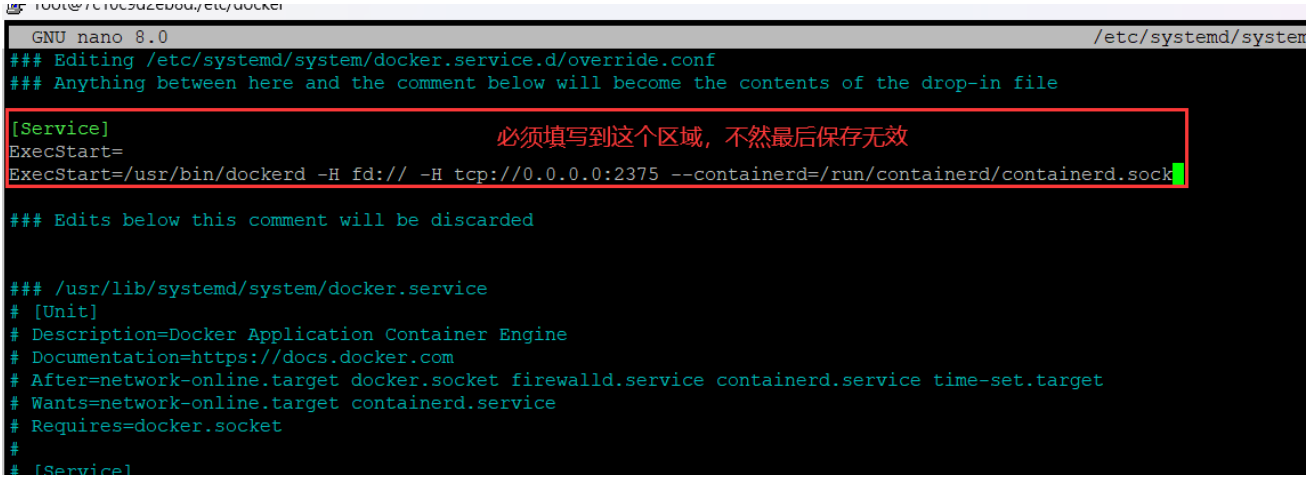
参考链接：https://docs.docker.com/config/daemon/remote-access/

使用下面命令编辑docker服务启动文件

```
systemctl edit docker.service
```

添加或修改以下行

```
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd -H fd:// -H tcp://0.0.0.0:2375 --
containerd=/run/containerd/containerd.sock
```



然后保存（ctrl+o保存、ctrl+x离开）文件，执行下面命令重新加载`systemctl`配置。

```
systemctl daemon-reload
```

重启Docker容器

```
systemctl restart docker
```

通过查看 netstat 的输出以确认 dockerd 正在侦听配置的端口，以检查更改是否已生效。

```
netstat -lntp | grep dockerd
```

```
[root@7c10c9d2eb8d docker]# netstat -lntp | grep dockerd
tcp6       0      0 :::2375                 :::*                    LISTEN      53038/dockerd
[root@7c10c9d2eb8d docker]#
```

开放防火墙端口

```
# 开放端口
firewall-cmd --add-port 2375/tcp --permanent
# 重新加载防火墙
firewall-cmd --reload
```

安全认证

由于开放了端口没有做任何安全保护，会引起安全漏洞，被人入侵、挖矿、CPU飙升这些情况都有发生。

为解决这个问题：我们只要使用安全传输层协议（TLS）进行传输并使用CA认证即可。

参考链接：https://docs.docker.com/engine/security/protect-access/

制作证书和秘钥

我们需要使用OpenSSL制作CA机构证书、服务端证书和客户端证书，以下操作均在安装Docker的Linux服务器上进行。

- 首先创建一个目录用于存储生成的证书和秘钥

```
mkdir /home/docker-ca && cd /home/docker-ca
```

- 创建CA证书私钥，期间需要输入两次用户名和密码，生成文件为`ca-key.pem`

```
openssl genrsa -aes256 -out ca-key.pem 4096
```

```
[root@localhost docker-ca]# openssl genrsa -aes256 -out ca-key.pem 4096
Generating RSA private key, 4096 bit long modulus (2 primes)
.....................................................................
.....................................................++++
.++++
e is 65537 (0x010001)
Enter pass phrase for ca-key.pem:
Verifying - Enter pass phrase for ca-key.pem:
```

密码根据自己情况输入就行，比如我输入都是123456。

- 根据私钥创建CA证书，期间需要输入上一步设置的私钥密码，生成文件为`ca.pem`

```
openssl req -new -x509 -days 365 -key ca-key.pem \
-sha256 -subj "/CN=*" -out ca.pem
```

```
[root@localhost docker-ca]# openssl req -new -x509 -days 365 -key ca-key.pem -sha256 -subj "/CN=*" -out ca.pem
Enter pass phrase for ca-key.pem:
[root@localhost docker-ca]#
```

- 创建服务端私钥，生成文件为`server-key.pem`

```
openssl genrsa -out server-key.pem 4096
```

```
[root@localhost docker-ca]# openssl genrsa -out server-key.pem 4096
Generating RSA private key, 4096 bit long modulus (2 primes)
.....................................++++
.....................................................................++++
e is 65537 (0x010001)
[root@localhost docker-ca]#
```

- 创建服务端证书签名请求文件，用于CA证书给服务端证书签名，生成文件`server.csr`

```
openssl req -subj "/CN=*" -sha256 -new \
-key server-key.pem -out server.csr
```

```
[root@localhost docker-ca]# openssl req -subj "/CN=*" -sha256 -new -key server-key.pem -out server.csr
[root@localhost docker-ca]#
```

- 创建CA证书签名好的服务端证书，期间需要输入CA证书私钥密码，生成文件为`server-cert.pem`

```
openssl x509 -req -days 365 -sha256 -in server.csr \
-CA ca.pem -CAkey ca-key.pem -CAcreateserial -out server-cert.pem
```

```
[root@localhost docker-ca]# openssl x509 -req -days 365 -sha256 -in server.csr -CA ca.pem -CAkey ca-key.pem -CAcreateserial -out server-cert.pem
Signature ok
subject=CN = *
Getting CA Private Key
Enter pass phrase for ca-key.pem:
[root@localhost docker-ca]#
```

- 创建客户端私钥，生成文件为`key.pem`

```
openssl genrsa -out key.pem 4096
```

```
[root@localhost docker-ca]# openssl genrsa -out key.pem 4096
Generating RSA private key, 4096 bit long modulus (2 primes)
.....................................................................................................................++++
.....................................................++++
e is 65537 (0x010001)
[root@localhost docker-ca]#
```

- 创建客户端证书签名请求文件，用于CA证书给客户证书签名，生成文件`client.csr`

```
openssl req -subj "/CN=client" -new -key key.pem -out client.csr
```

```
[root@localhost docker-ca]# openssl req -subj "/CN=client" -new -key key.pem -out client.csr
[root@localhost docker-ca]#
```

- 为了让秘钥适合客户端认证，创建一个扩展配置文件`extfile-client.cnf`

```
echo extendedKeyUsage = clientAuth > extfile-client.cnf
```

```
[root@localhost docker-ca]# echo extendedKeyUsage = clientAuth > extfile-client.cnf
[root@localhost docker-ca]#
```

- 创建CA证书签名好的客户端证书，期间需要输入CA证书私钥密码，生成文件为`cert.pem`

```
openssl x509 -req -days 365 -sha256 -in client.csr \
-CA ca.pem -CAkey ca-key.pem -CAcreateserial \
-out cert.pem -extfile extfile-client.cnf
```

```
[root@localhost docker-ca]# openssl x509 -req -days 365 -sha256 -in client.csr \
> -CA ca.pem -CAkey ca-key.pem -CAcreateserial \
> -out cert.pem -extfile extfile-client.cnf
Signature ok
subject=CN = client
Getting CA Private Key
Enter pass phrase for ca-key.pem:
[root@localhost docker-ca]#
```

- 删除创建过程中多余的文件

```
rm -rf ca.srl server.csr client.csr extfile-client.cnf
```

- 最终生成文件如下，有了它们我们就可以进行基于TLS的安全访问了

```
[root@localhost docker-ca]# ls
ca-key.pem  ca.pem  cert.pem  key.pem  server-cert.pem  server-key.pem
[root@localhost docker-ca]# ll
```

```
ca.pem CA证书
ca-key.pem CA证书私钥
server-cert.pem 服务端证书
server-key.pem 服务端证书私钥
cert.pem 客户端证书
key.pem 客户端证书私钥
```

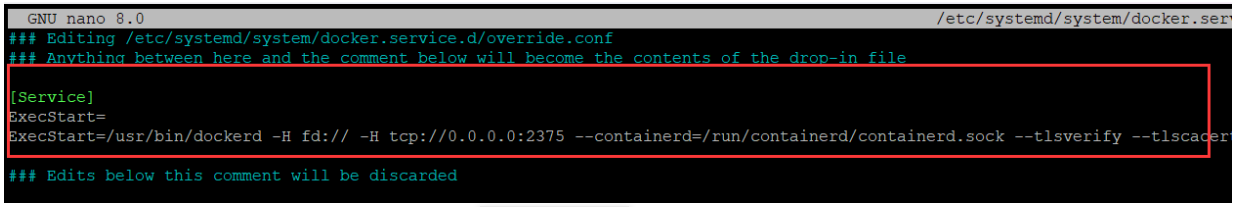
配置Docker支持TLS

- 编辑docker服务启动文件

```
systemctl edit docker.service
```

- 在原来的基础上往后添加启动参数

```
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd -H fd:// -H tcp://0.0.0.0:2375 --containerd=/run/containerd/containerd.sock --tlsverify --tlscacert=/home/docker-ca/ca.pem --tlscert=/home/docker-ca/server-cert.pem --tlskey=/home/docker-ca/server-key.pem
```

- 

- 然后保存文件，执行下面命令重新加载`systemctl`配置。

```
systemctl daemon-reload
```

- 重启Docker容器

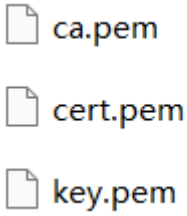
```
systemctl restart docker
```

客户端访问

下载证书到本地磁盘，主要下载下面三个文件

```
ca.pem CA证书
cert.pem 客户端证书
key.pem 客户端证书私钥
```

比如我下载到我的本地电脑的`E:\Workspace\docker-ca`这个位置。



记住这个位置，后面打包部署的时候需要指定改目录的，具体使用我们在打包部署的时候会用到。

Docker命令

进程相关命令

```
# 启动Docker命令
systemctl start docker
# 停止Docker命令
systemctl stop docker
# 重启Docker命令
systemctl restart docker
# 查看启动结果
systemctl status docker
# 设置开机启动Docker
systemctl enable docker
```

镜像相关命令

我们可以直接输入 docker 命令来查看到 Docker 客户端的所有命令选项

可以通过命令 **docker command --help** 更深入的了解指定的 Docker 命令使用方法。

例如我们要查看 **docker stats** 指令的具体使用方法：

```
docker stats --help
```

查看镜像

查看本地所有的镜像

```
# 列出本地主机上的镜像
docker images
# 查看所用镜像的id
docker images -q
```

搜索镜像

从网络中查找需要的镜像

```
docker search 镜像名称
```

拉取镜像

从Docker仓库下载镜像到本地，镜像名称格式为 名称:版本号，如果版本号不指定则是最新的版本。

如果不知道镜像版本，可以去 Docker Hub搜索对应镜像查看。

```
docker pull 镜像名称:版本号
# 完成命令 docker pull [选项] [Docker Registry 地址[:端口号]/]仓库名[:标签]
```

具体的选项可以通过 `docker pull --help` 命令看到，这里我们说一下镜像名称的格式。
Docker 镜像仓库地址：地址的格式一般是 <域名/IP>[:端口号] 。默认地址是 Docker Hub。
仓库名：如之前所说，这里的仓库名是两段式名称，即 <用户名>/<软件名> 。对于 Docker Hub，如果不给出用户名，则默认为 library ，也就是官方镜像。

如：

```
docker pull centos:7
```

上面的命令中没有给出 Docker 镜像仓库地址，因此将会从 Docker Hub 获取镜像。而镜像名称是 centos:7 ，因此将会获取官方镜像 library/centos 仓库中标签为 7的镜像。

从下载过程中可以看到我们之前提及的分层存储的概念，镜像是由多层存储所构成。下载也是一层层的去下载，并非单一文件。下载过程中给出了每一层的 ID 的前 12 位。并且下载结束后，给出该镜像完整的 sha256 的摘要，以确保下载一致性。

删除镜像

删除本地镜像

```
# 删除指定本地镜像
docker rmi 镜像id #这里也可以是<镜像名:tag>
# 删除所有本地镜像
docker rmi `docker images -q`
```

容器相关命令

创建容器

```
docker run [参数] 镜像名称:Tag <交互方式>
```

参数说明：

- -i：保持容器运行。通常与 -t 同时使用。加入it这两个参数后，容器创建后自动进入容器中，退出容器后，容器自动关闭。
- -t：为容器重新分配一个伪输入终端，通常与 -i 同时使用。
- -d：以守护（后台）模式运行容器。创建一个容器在后台运行，需要使用docker exec 进入容器。退出后，容器不会关闭。
- -it 创建的容器一般称为交互式容器，-id 创建的容器一般称为守护式容器
- --name：为创建的容器命名。
- /bin/bash:交互方式为shell终端

命令示例：

```
# 创建容器c1并进入
docker run -it --name c1 centos:7 /bin/bash
# 创建容器并后台启动
docker run -id --name c2 centos:7
```

查看容器

```
# 查看正在运行的容器
docker ps
# 查看所有容器
docker ps -a
# 查看所有容器id
docker ps -aq
```

进入容器

```
docker exec [参数] 容器名称 /bin/bash
```

命令示例：

```
# 进入c2容器
docker exec -it c2 /bin/bash
```

启动容器

```
docker start 容器名称
```

命令示例：

```
# 启动c1容器
docker start c1
```

停止容器

```
docker stop 容器名称
```

命令示例：

```
# 停止c1容器
docker stop c1
```

删除容器

如果容器是运行状态则删除失败，需要停止容器才能删除

```
# 删除指定容器
docker rm 容器名称
# 删除所用容器
docker rm `docker ps -aq`
```

查看容器信息

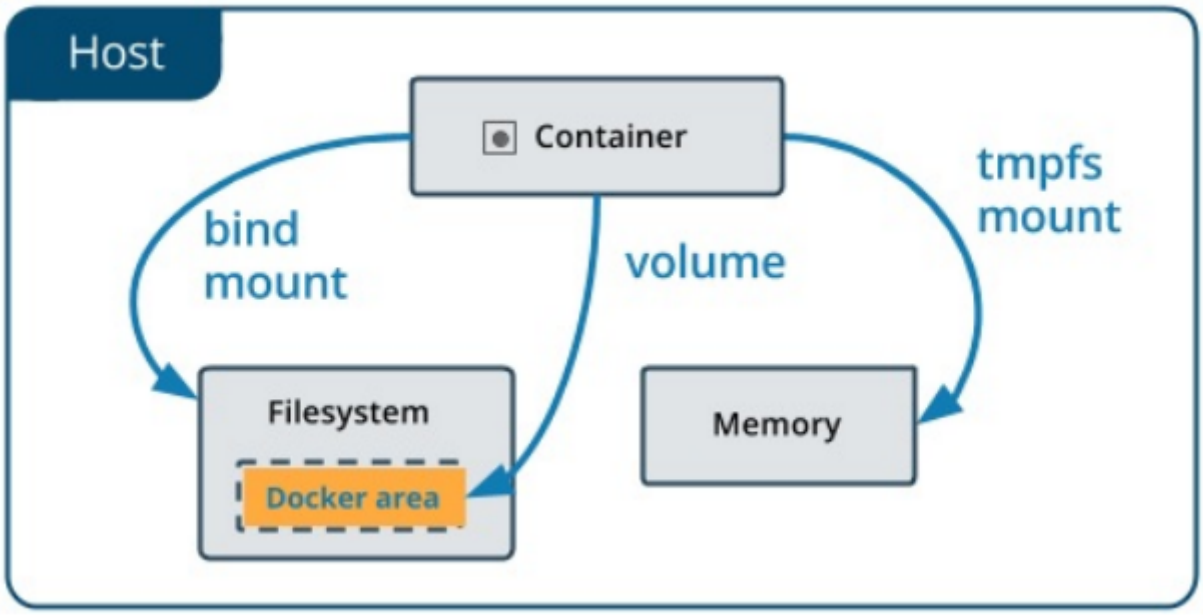
```
docker inspect 容器名称
```

更新容器配置

指令：`docker update [OPTIONS] CONTAINER [CONTAINER ...]`

```
# 演示设置容器跟随docker一起重启
docker update --restart always 容器名称
```

Docker 数据管理



数据卷

数据卷的概念

- 数据卷是宿主机中的一个目录或文件
- 当容器目录和数据卷目录绑定后，对方的修改会立即同步
- 一个数据卷可以被多个容器同时挂载
- 一个容器也可以被挂载多个数据卷

- 数据卷，默认会一直存在，即使容器被删除

数据卷的作用

- 容器数据持久化
- 外部机器和容器间接通信
- 容器之间数据交换

配置数据卷

创建启动容器时，使用 –v 参数 设置数据卷

```
docker run ... -v 宿主机目录(文件):容器内目录(文件) ... 镜像名:tag [交互方式]
```

注意事项：

- 目录必须是绝对路径
- 如果目录不存在，会自动创建
- 可以挂载多个数据卷

命令示例

```
#挂载单目录
docker run -it --name c1 -v /root/volume:/root/volum_container centos:7
#挂载多目录
docker run -it --name c2 -v /root/data:/root/data -v /root/data1:/root/data1
centos:7
```

数据卷容器

多容器进行数据交换

- 多个容器挂载同一个数据卷(弊端：如果容器多了，创建容器的时候构建数据卷比较麻烦)
- 数据卷容器

数据卷容器实现步骤

创建数据卷容器，使用-v指定数据卷

```
docker run -id --name 容器名称 -v 宿主机目录路径 镜像名:Tag
```

为容器设置数据卷容器，使用--volumes-from设置数据卷容器

```
docker run -id --name 容器名称 --volumes-from 数据卷容器名 镜像名:Tag
```